1. Introduction to (Large) Language Models

CSC3043S Reasoning in AI: Large Language Models

Dr Jan Buys

University of Cape Town



UNIVERSITY OF CAPE TOWN

IYUNIVESITHI YASEKAPA • UNIVERSITEIT VAN KAAPSTAD

CSC3043S AI: Reasoning in AI

Convenor:

- Dr. Jan Buys (jan.buys@uct.ac.za)

TA:

- Christopher Allpass-Jackson (ALLCHR011@myuct.ac.za)

- **Prerequisites**: CSC2001F, CSC2041F, CSC2042S, CSC3042F
- Lectures will be in the third term only
- See the course notes for details on the course structure and assessments

CSC3043S AI: Reasoning in AI

The course consists of 3 topics, with 2 weeks of lectures each:

- Large Language Models – Dr Jan Buys
- Vision language models and visual generative models – Prof Patrick Marais
- Bayesian networks – Dr Jan Buys

Large Language Models

Lecture structure:

Date	Monday	Tuesday	Wednesday	Thursday	Friday
27-Jul	Introduction to LLMs	Tokenization	Transformer architecture for LLMs	LLM Training optimization	Tutorial: LLM training implementation
3-Aug	LLM Inference	Instruction fine-tuning	LLM Alignment	Reasoning and test-time scaling	Tutorial: Post-training

Assignment: Training a (small) language model from scratch

Large Language Models (from scratch)

- This course is about the technical fundamentals underlying modern language models, not about how to use LLMs for particular applications
- This is a large topic with a lot of advanced details that are out of the scope of this course, but we will cover the most important concepts related to the different stages of developing and applying LLMs
- We will touch on concepts related to Natural Language Processing (NLP), but this is not an NLP course
- The ultimate goal is to understand how LLMs have become AI reasoning models

Course Material

Slides and other resources will be provided with each lecture. The majority of the content is covered in:

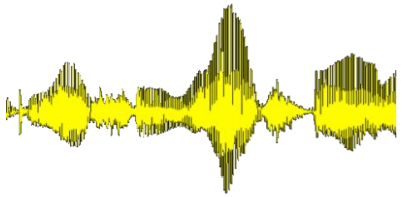
- Speech and Language Processing (3rd ed. draft). Dan Jurafsky and James H. Martin. Available at: https://web.stanford.edu/~jurafsky/slp3/

There are many tutorials, blog posts etc. online!

(Natural) Language

What is special about human language?

- A human language is a system specifically constructed to convey the speaker/writer's meaning
- A human language is a **discrete/symbolic/categorical signaling system**
  - rocket = 🚀; violin = 🎻
  - With very minor exceptions for expressive signaling (e.g. "I loooove it.", "Whoomppaaa")
- The categorical symbols of a language can be encoded as a signal for communication in several ways: Sound, gesture, images (writing)
  - The **symbol** is **invariant** across different encodings

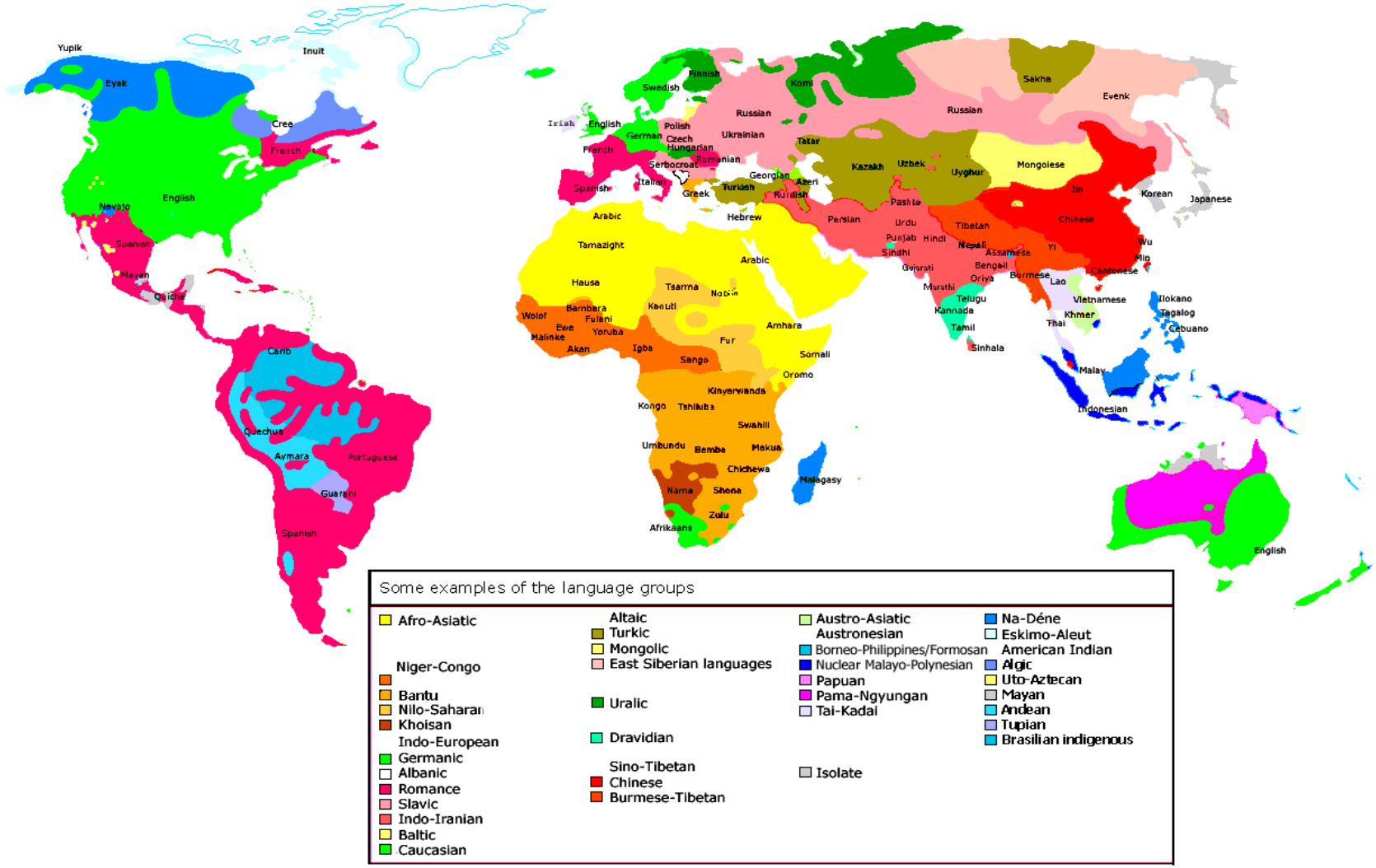






Slide: Chris Manning (Stanford CS224n)

Natural Languages



Natural Language

- Language consists of sequences of *words* that are organized into *sentences*
  - The *morphology* of a language determines how words are formed
    - E.g. “dogs” = “dog” + “s”
    - E.g. “Ndiyabulela” = “Ndi” + “ya” + “bulela” (I am grateful)
  - The *syntax* of a language determines which sequences of words are valid sentences (regardless of meaning)
    - E.g. “Colorless green ideas sleep furiously” vs “Furiously sleep ideas green colorless”

Natural Language

- Language consists of sequences of *words* that are organized into *sentences*
  - *Semantics* refers to the meaning of sentences
    - Traditionally, natural language semantics is represented by logics or graphs
    - Distributional hypothesis: "You shall know a word by the company it keeps.” Words occurring in similar contexts have similar meaning
    - “The best representation of natural language is natural language”

Natural Language Processing

Natural Language Processing is the study of systems that process human language and enable computers to perform useful tasks involving human language

- The fundamental goal is *deep understand* of *general-purpose* language, not just string processing or keyword matching

Main components of NLP:

- Analysis/Understanding (NL $\rightarrow \mathcal{R}$)
- Generation ($\mathcal{R}\rightarrow$ NL)
- Acquisition of the representation ($\mathcal{R}$) from knowledge and data

$\mathcal{R}$ is a representation that is useful for a computer. It might or might not be interpretable to humans or based on scientific theories.

- Traditional NLP often used explicit linguistic representations (Parts-of-Speech, syntactic trees, semantic graphs)
- Current approaches are based on vector representations of letters, words, or text passages

Slide: Noah Smith (UW CSE517)

Representations in NLP

- Word vectors

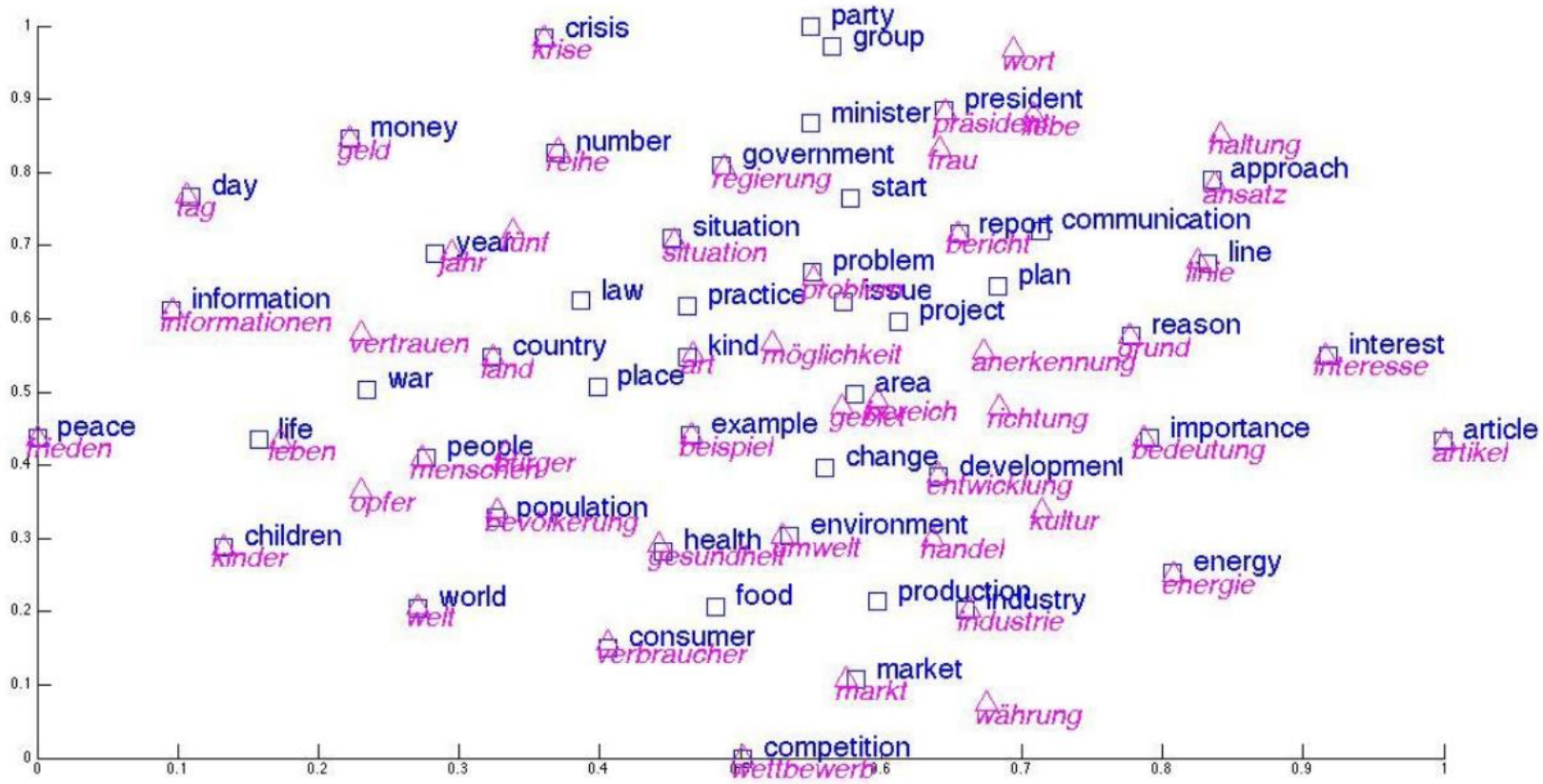


Two-dimensional (t-SNE) projection of embeddings

Jurafsky and Martin (2020) Ch. 6

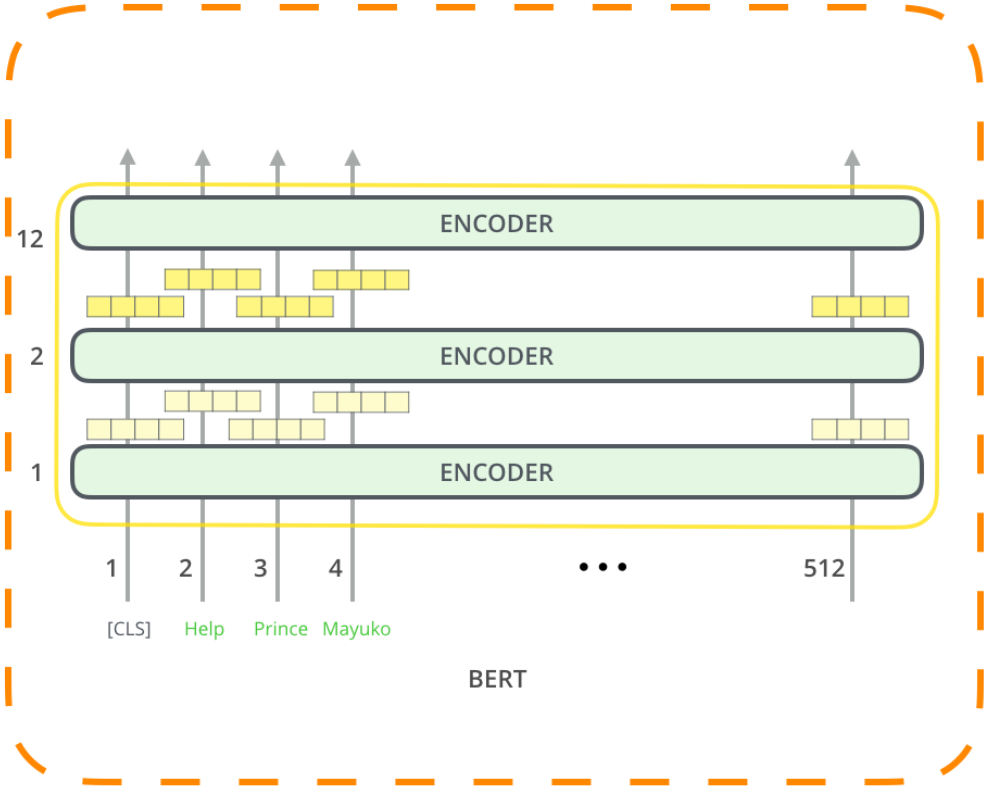
Representations in NLP

- Multilingual modelling: Word embeddings multiple languages



Representations in NLP

- Contextualized word representations in LLMs: Text encoder with sequence models such as Transformers



http://jalammar.github.io/illustrated-bert/

NLP tasks

- Traditional, separate models were developed for each task

Understanding	Generation
Sentiment analysis	Machine Translation
Topic classification	Summarization
Named Entity Recognition	Data-to-text generation
Question answering	

- Now, all of these, as well as tasks that were not considers to be part of NLP, such as code generation or mathematical reasoning, can be handled by Large Language Models

NLP is hard problem

NLP is a hard problem for computers (or at least was before LLMs) for multiple reasons:

- Ambiguity: words and sentences may have many possible syntactic analyses, and many possible meanings (semantics) which depends on the context
- Formalizing linguistic rules was not a scalable approach to NLP due to the complexity of linguistic structure and the long tail of exceptions to each rule
- Sparsity: A few words are used very frequently, but many more are used infrequently, and many valid combinations won't occur even in web-scale corpora

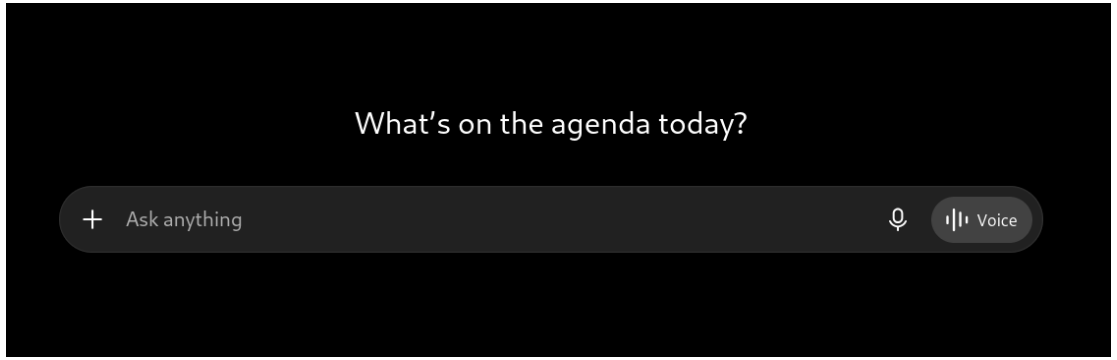
Other challenges include richness and diversity of language, robustness, and scalability of algorithms

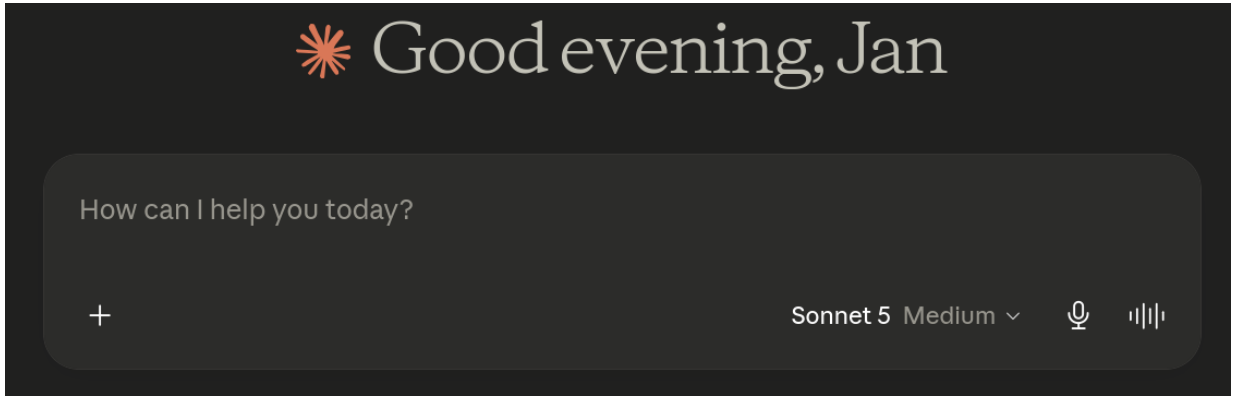
Brief History of NLP and LLMs

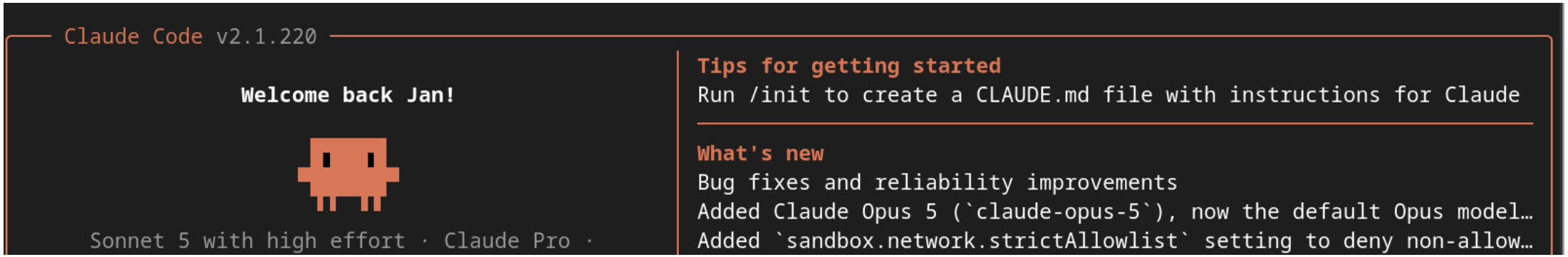
- Machine learning approaches replaced earlier rule-based approaches in the 1990s/2000s, but linguistic resources were still widely used and systems often consisted of pipelines with intermediate linguistic analyses
- In the 2010's, neural network-based models (deep learning) became widely used and were scaled up to be "end-to-end" systems trained on large datasets without making intermediate linguistic predictions
- Currently (2020s) the dominant approach in NLP research is using Large Language Models – neural network models (in particular based on the Transformer architecture) that were trained on very large text corpora to learn general patterns from text
- LLMs now form the basis of a lot of AI beyond traditional NLP

J&M Chapter 1

Language models







Language models



- Language models predict the next word in a word sequence

Language Modelling

- Suppose we have a document consisting of *n* word **tokens** in a language that where *V* is the set of word **types**
- A **language model** is a probabilistic model of a document

$$P(w_1, w_2, \ldots, w_n;\ \theta)$$

- We can use the **chain rule** to decompose the probability distribution:

$$P(w_1 w_2 \ldots w_n) = \prod_i P(w_i | w_1 w_2 \ldots w_{i-1})$$

- An **autoregressive language model** predicts the next token conditioned on the previous tokens

Slide: Sasha Rush

Language Modelling

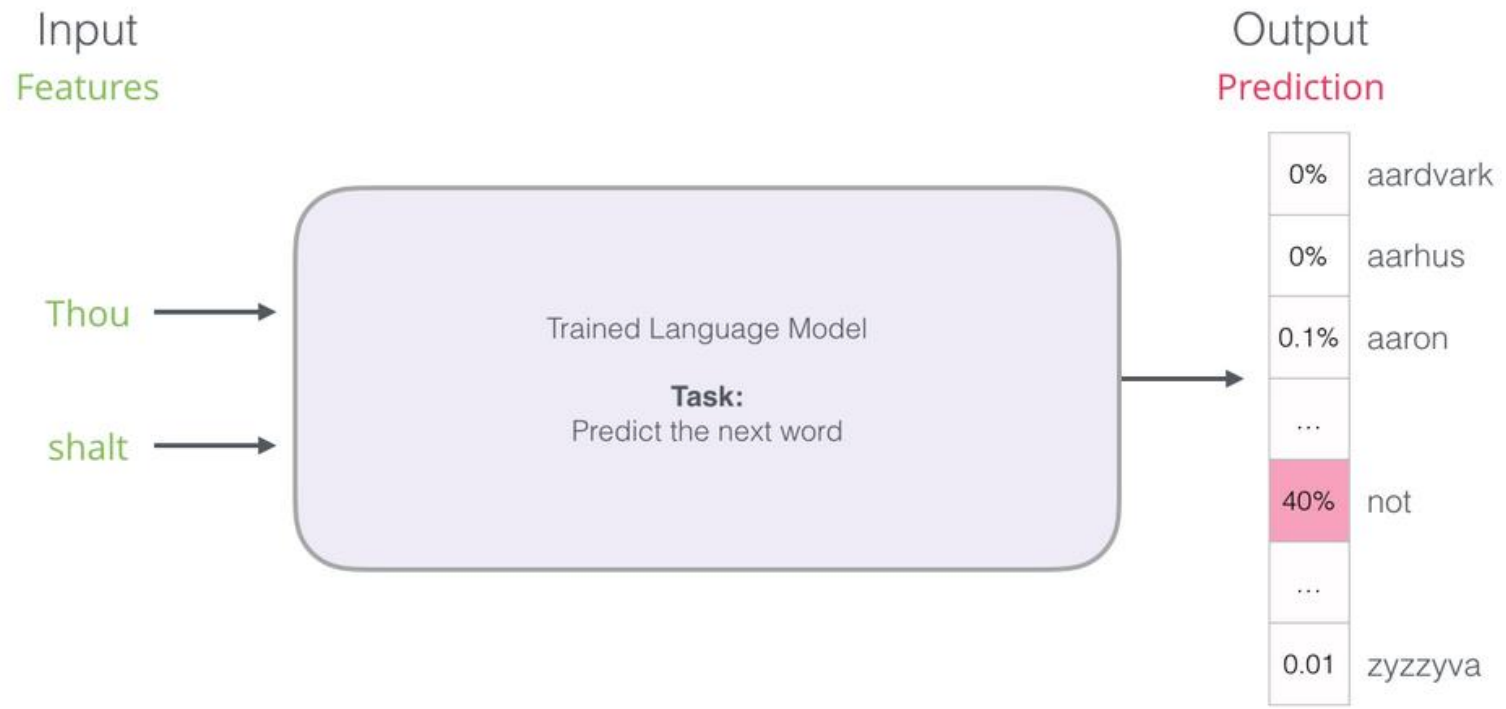
- Predict the next word in a word sequence



https://jalammar.github.io/illustrated-word2vec/

Language Modelling

- Assign a probability to the next word



https://jalammar.github.io/illustrated-word2vec/

Language Modelling

- Generate text by sampling one word at a time:

$$x_1 \sim p(x_1)$$

$$x_2 \sim p(x_2 \mid x_1)$$

$$\vdots$$

$$x_T \sim p(x_T \mid x_1, \ldots, x_{T-1})$$

Slide: Sasha Rush

Language Modelling

Classical view: Language models are used to model fluency

- One component of NLP models, applied via Bayes' rule

$$P(output\,|input) \propto P(input \mid output)\,P(output)$$

- Example applications:
  - Machine Translation
    - P("**high** winds tonight") > P("**large** winds tonight")
  - Spell Correction
    - P("about fifteen **minutes** from") > P("about fifteen **minuets** from")
  - Speech Recognition
    - P("recognize speech") > P("wreck a nice beach")
    - P("I saw a van") >> P("eyes awe of an")
    - P("I ate a cherry") >> P("eye eight uh Jerry")

Language Modelling

Modern view: Language models

- are used directly to generate text
- can recall factual knowledge from the training data
- can perform reasoning
- can execute instructions when prompted
- can condition on multimodal inputs (e.g. images) and generate not only language text but code, commands to be executed, etc.

What kinds of things does LM training learn?

- Stanford University is located in _____, California. [Trivia]
- I put ___ fork down on the table. [syntax]
- The woman walked across the street, checking for traffic over ___ shoulder. [coreference]
- I went to the ocean to see the fish, turtles, seals, and _____. [lexical semantics/topic]
- Overall, the value I got from the two hours watching it was the sum total of the popcorn and the drink. The movie was ___. [sentiment]
- Iroh went into the kitchen to make some tea. Standing next to Iroh, Zuko pondered his destiny. Zuko left the _____. [some reasoning – this is harder]
- I was thinking about the sequence that goes 1, 1, 2, 3, 5, 8, 13, 21, ____ [some basic arithmetic]

Slide from Jesse Mu

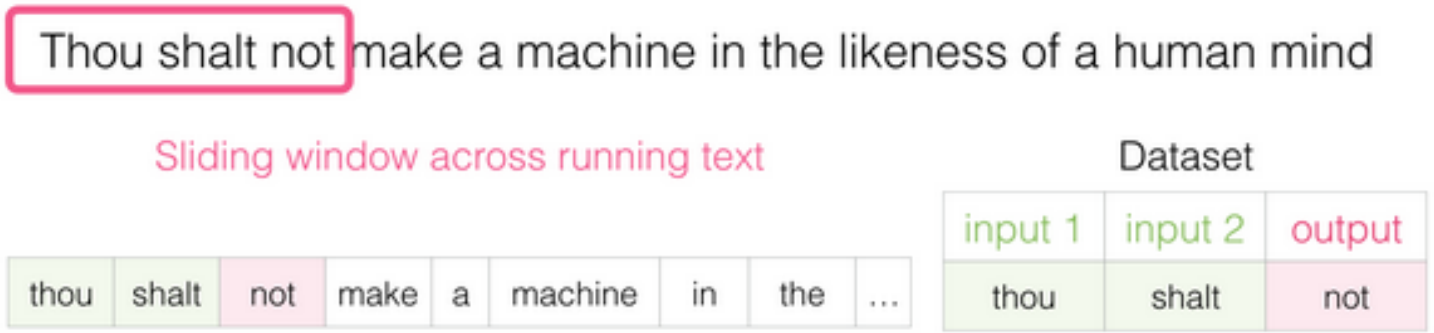
N-gram Language Models

- Traditional language models were based on count-based probability estimates rather than neural networks
- N-gram models uses a fixed context length, e.g. a 5-gram model conditions on the 4 previous words to predict the next (5th) word
- *n*-gram model: use a fixed length sliding window conditioning on the previous *N - 1* tokens

$$P(w_t|w_1^{t-1}) \approx P(w_t|w_{t-N+1}^{t-1})$$

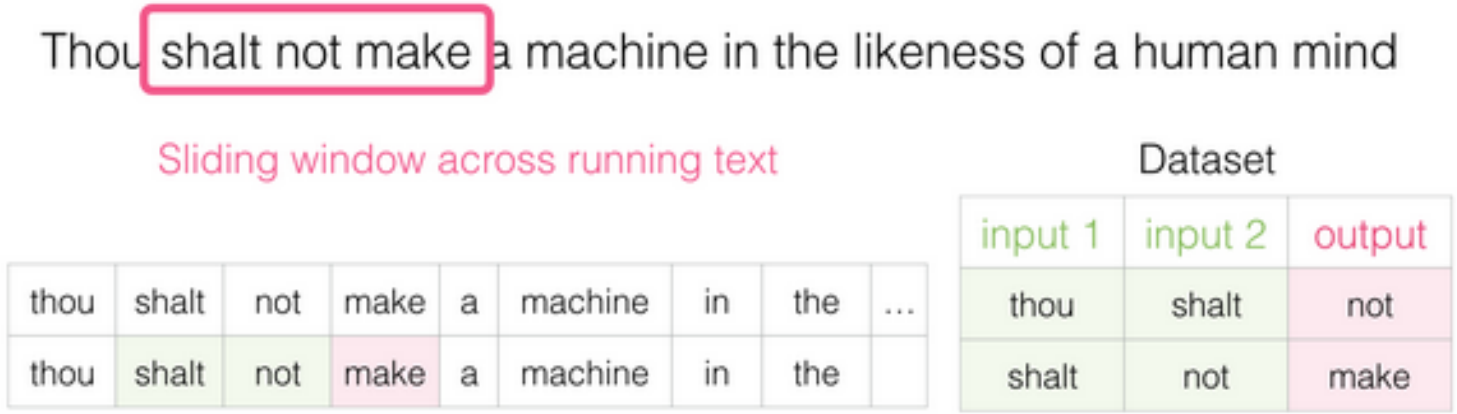
Jurafsky and Martin (2020) Ch. 9

N-gram Language Models



https://jalammar.github.io/illustrated-word2vec/

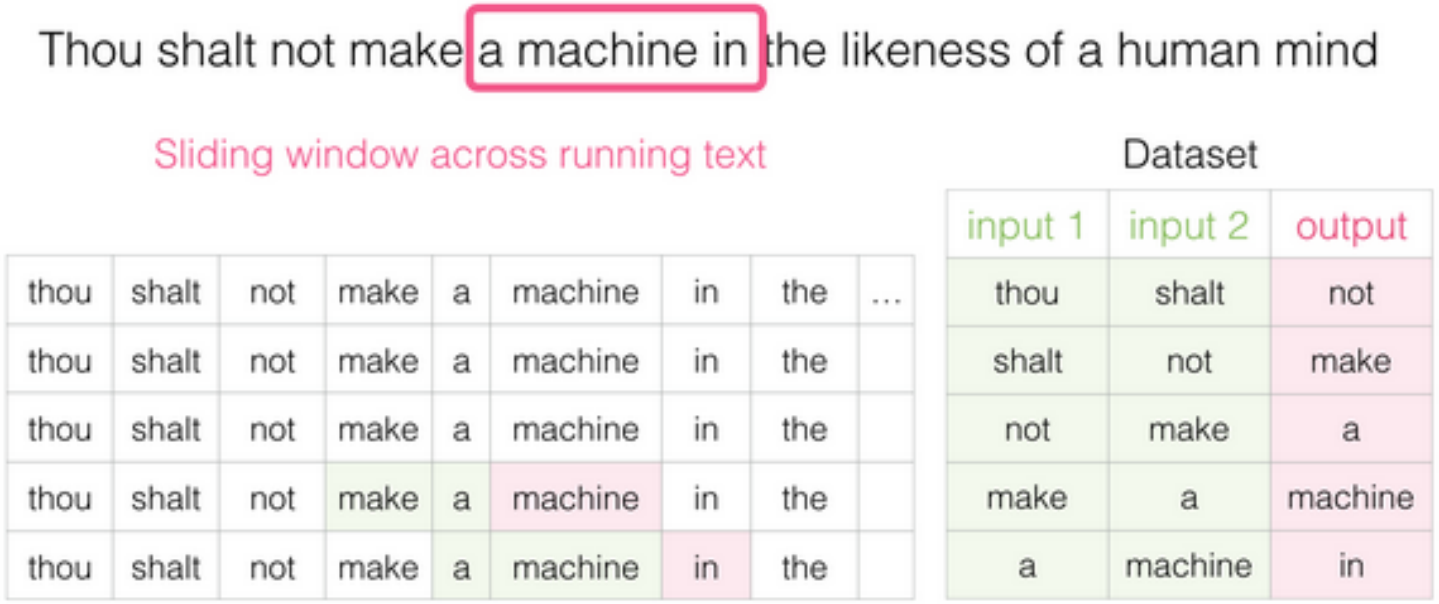
N-gram Language Models



https://jalammar.github.io/illustrated-word2vec/

N-gram Language Models

- Sliding window approach to construct language model training data and to apply the model to score n-grams



https://jalammar.github.io/illustrated-word2vec/

N-gram Language Models

- The problem with count-based estimates is that even in a very large corpus, most possible *n*-grams will never appear
- However models need some way to assign a non-zero probability to each *n*-gram

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

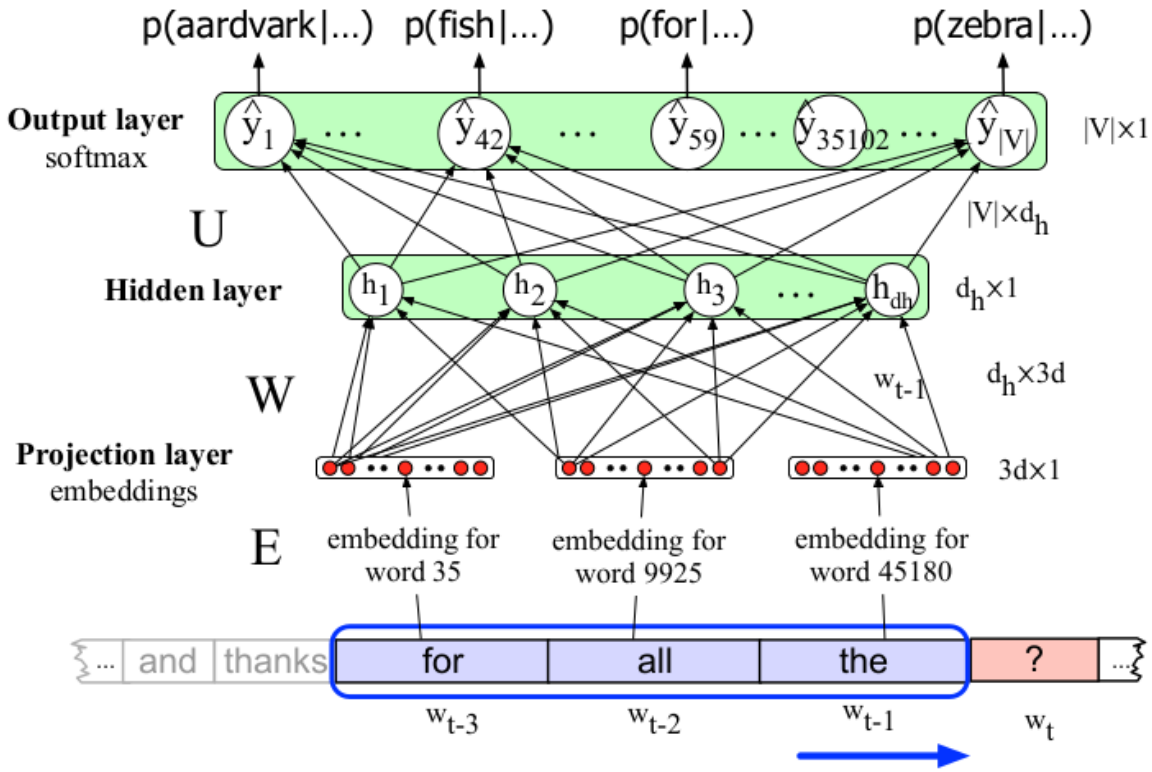
Jurafsky and Martin (2020)

N-gram Language Models

- Complex formulas were used to interpolate probabilities between different *n*-gram sizes (context lengths) and to estimate the probability of unseen *n*-grams by “smoothing” the probability distribution
- The state-of-the-art method was modified Kneser-Ney smoothing
- The advantage of this approach is that it is very computationally efficient, so fast to train on large datasets and can be used to score many candidate word sequences when used in language generation models such as for statistical machine translation
- However, due to sparsity in language they couldn’t scale up to using long contexts effectively even when trained on > 1 trillion tokens

Language modelling architectures

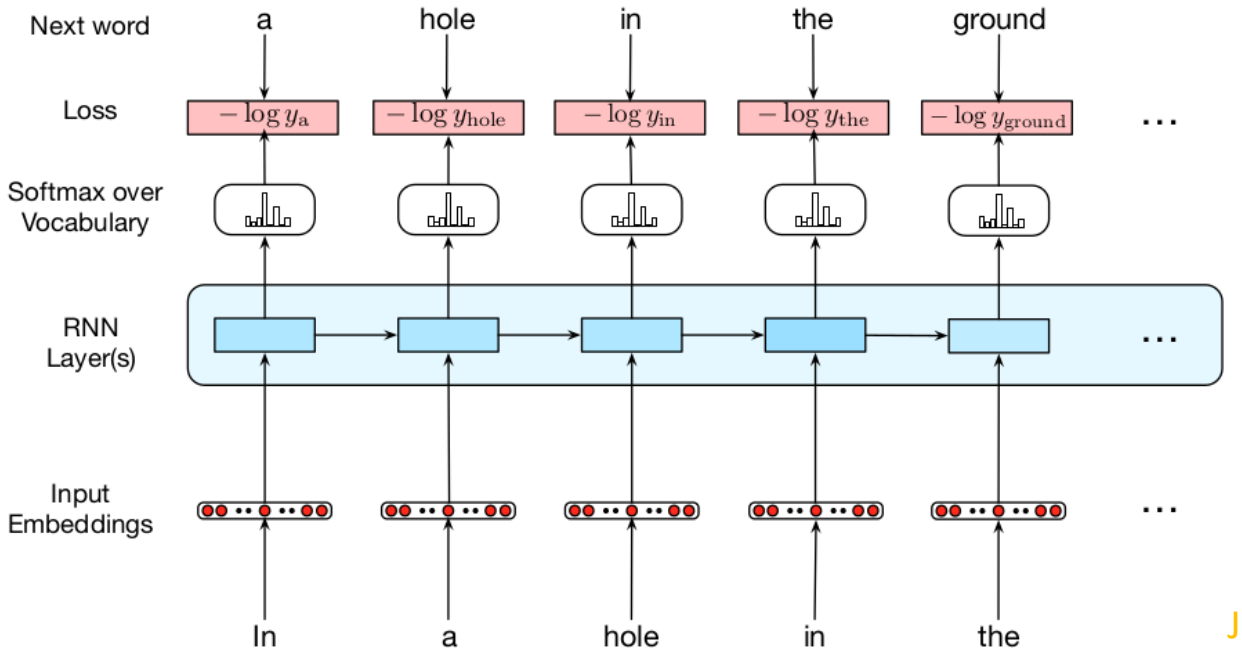
Feed-forward neural networks can be used to estimate language model probabilities, but suffer from many of the same limitations as count-based *n*-gram models such as supporting a relatively small context



Jurafsky and Martin (2020)

Language modelling architectures

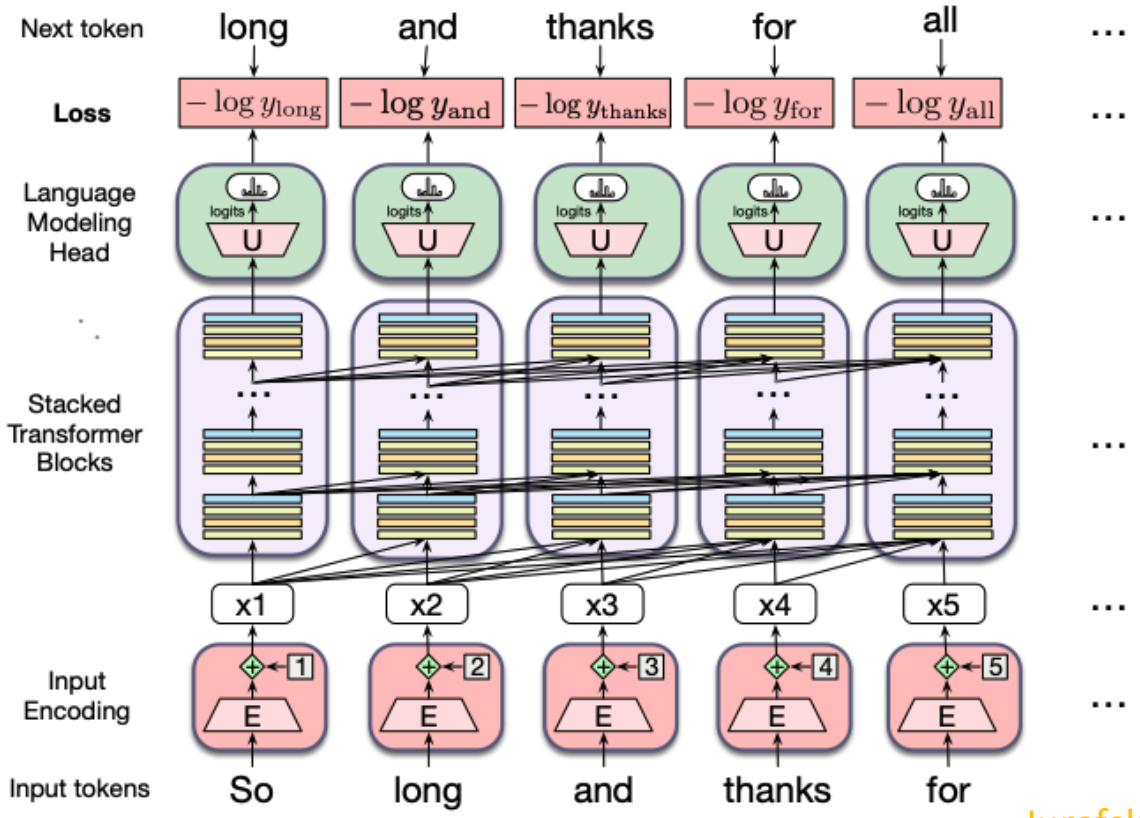
- Recurrent neural networks model sequences directly without an explicit context length limitation
- LSTM recurrent neural networks were the first widely used deep learning-based language models



Jurafsky and Martin (2020)

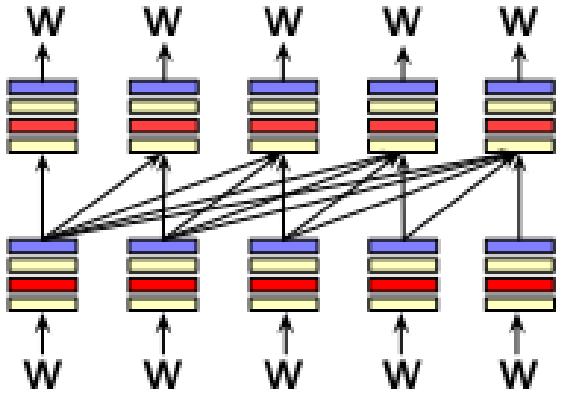
Language modelling architectures

- The Transformer architecture proved to be more scalable than LSTMs and became the dominant language model architecture

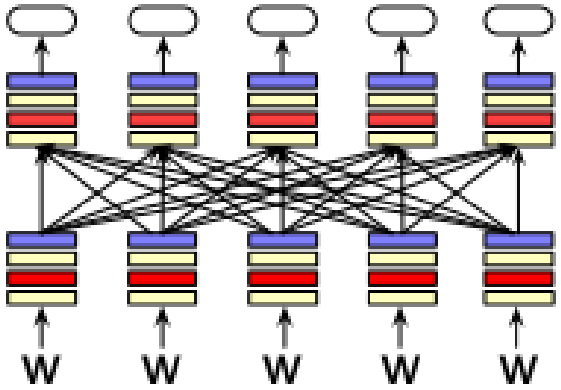


Jurafsky and Martin (2026) Ch. 8

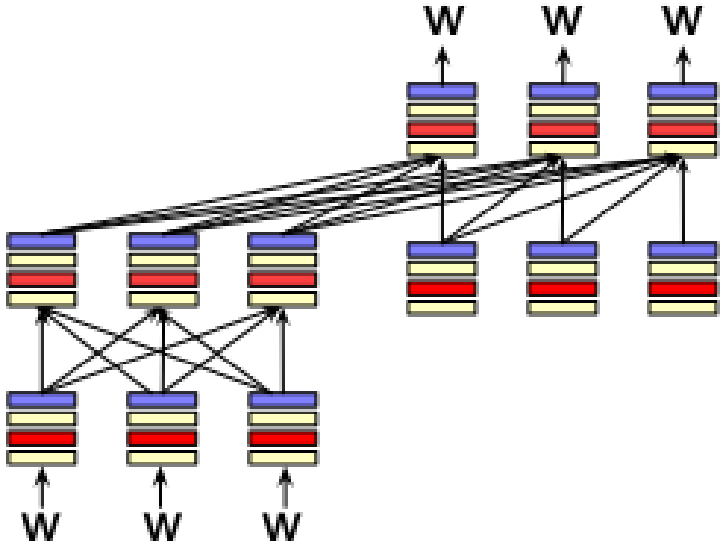
Three topologies for LLMs



Decoders



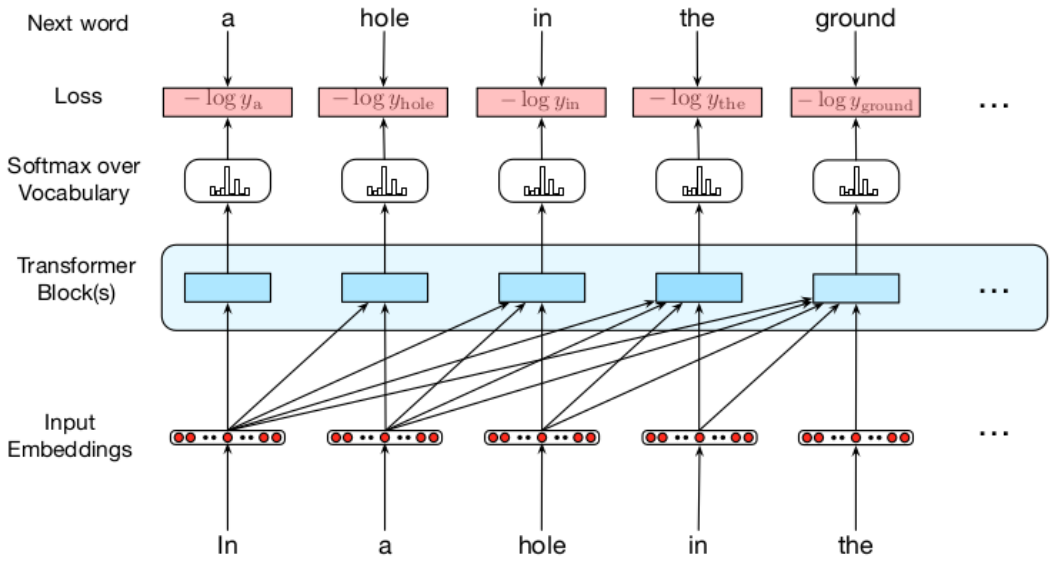
Encoders



Encoder-decoders

Decoder LLMs

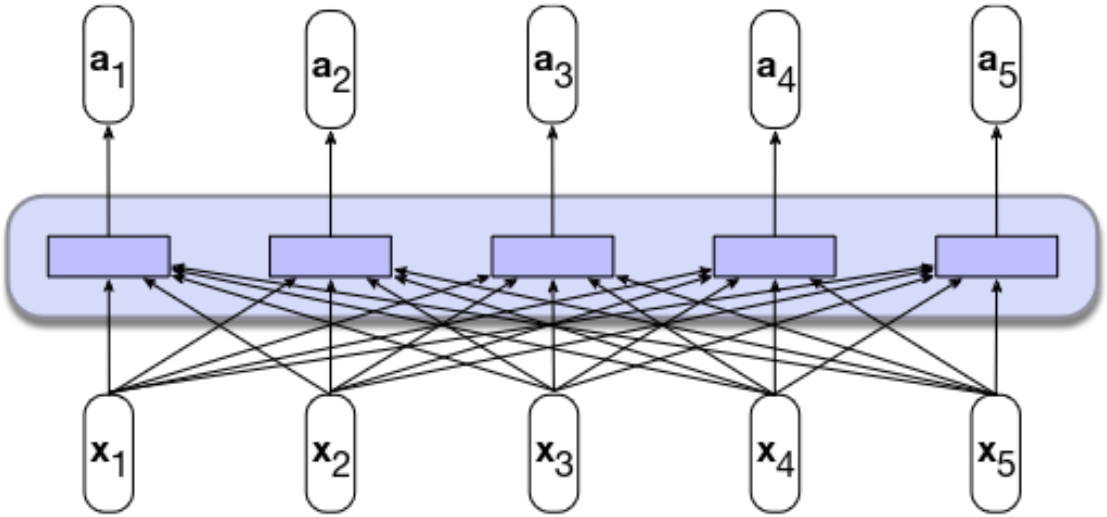
- Autoregressive language models as defined above, trained for next word prediction
- Transformer attention uses “causal” mask to only condition on past tokens when predicting the text token
- Used by most LLMs, but not the only option



Jurafsky and Martin (2020)

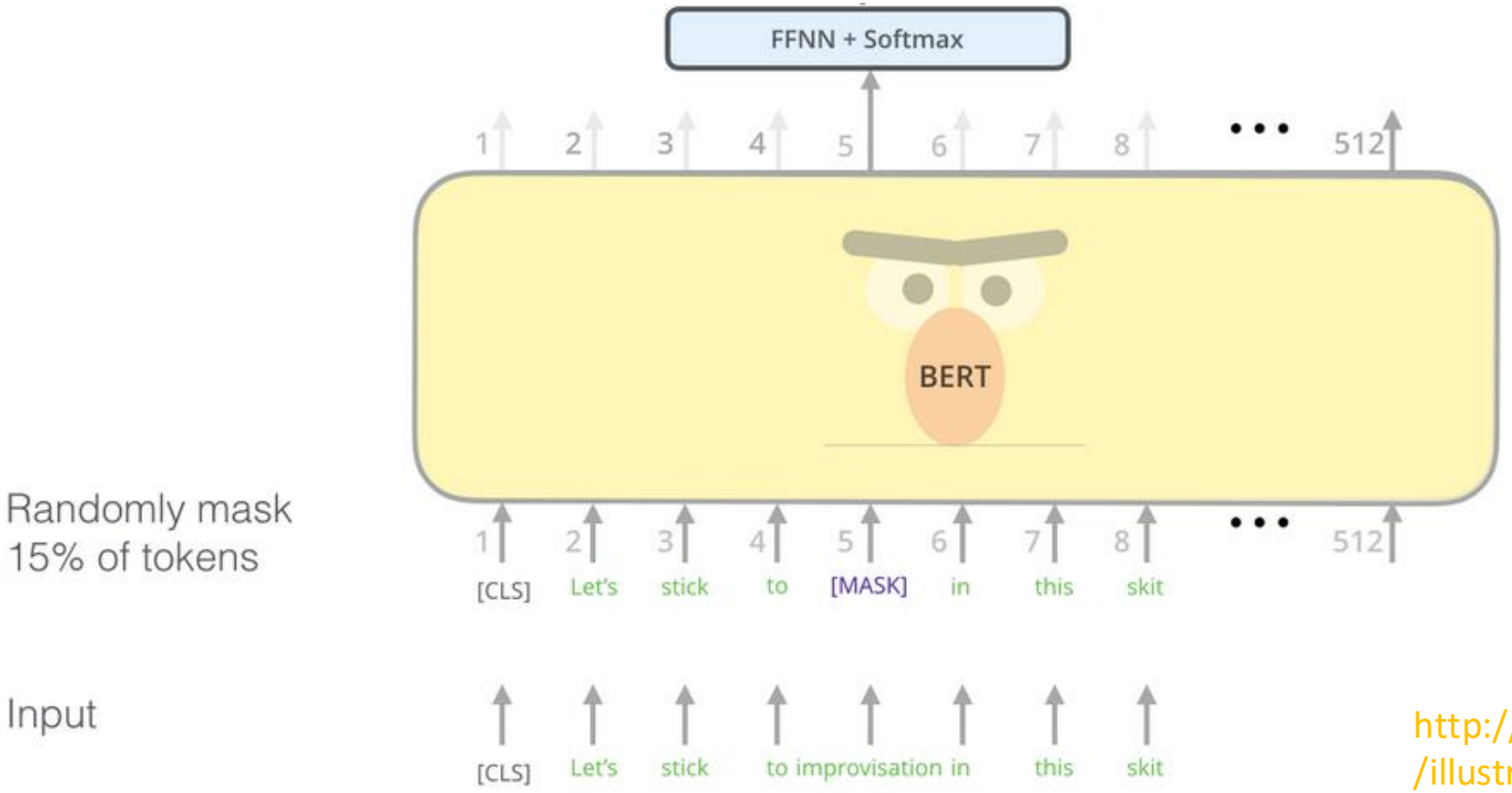
Encoder LLMs

- Bidirectional Encoder Representations from Transformers (BERT)
  - Devlin et al, 2018: First research to show that pretraining a Transformer on a large corpus can lead to better performance on NLP tasks than task-specific training only
  - Only encodes text, rather than generating, and uses context from both the left and right of a token, rather than the causal attention mask of decoders



Encoder LLMs

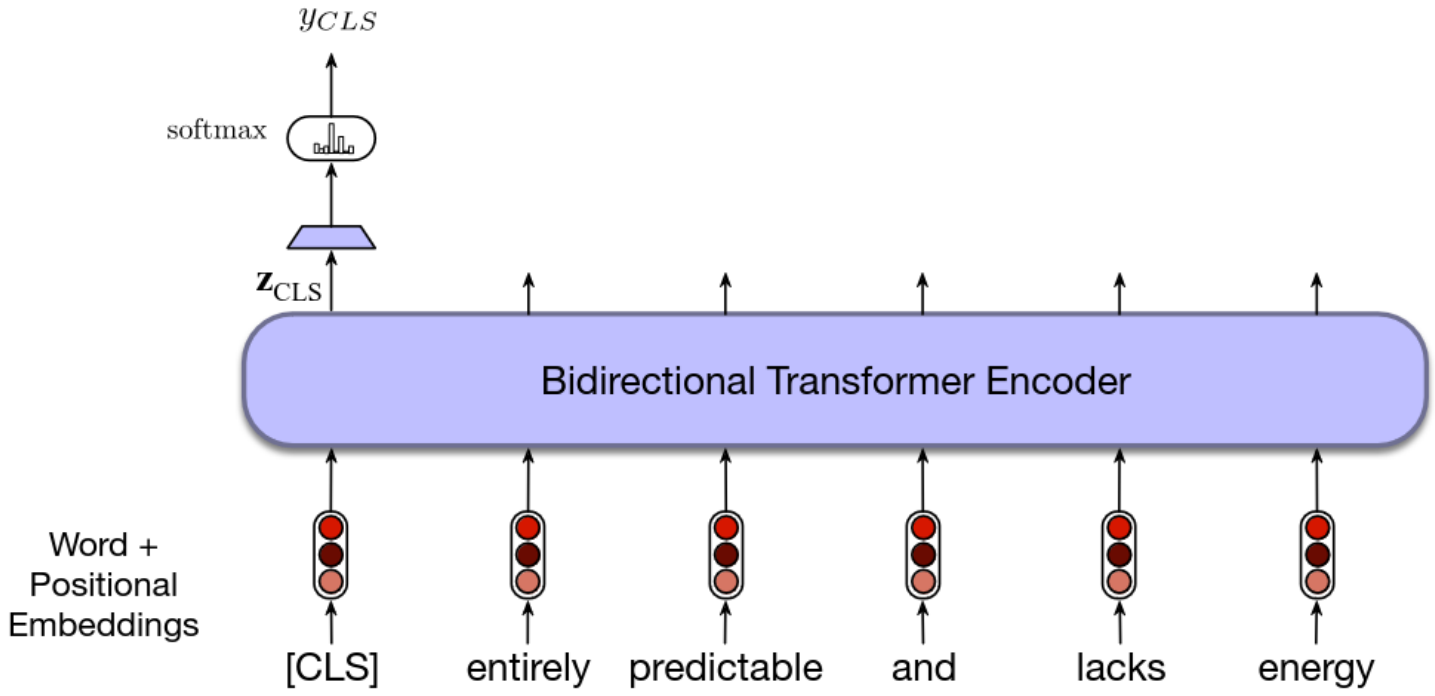
- BERT: Pretraining with *masked* language modelling objective
  - Repredict a random subset of tokens, using context from both sides



http://jalammar.github.io/illustrated-bert/

Encoder LLMs

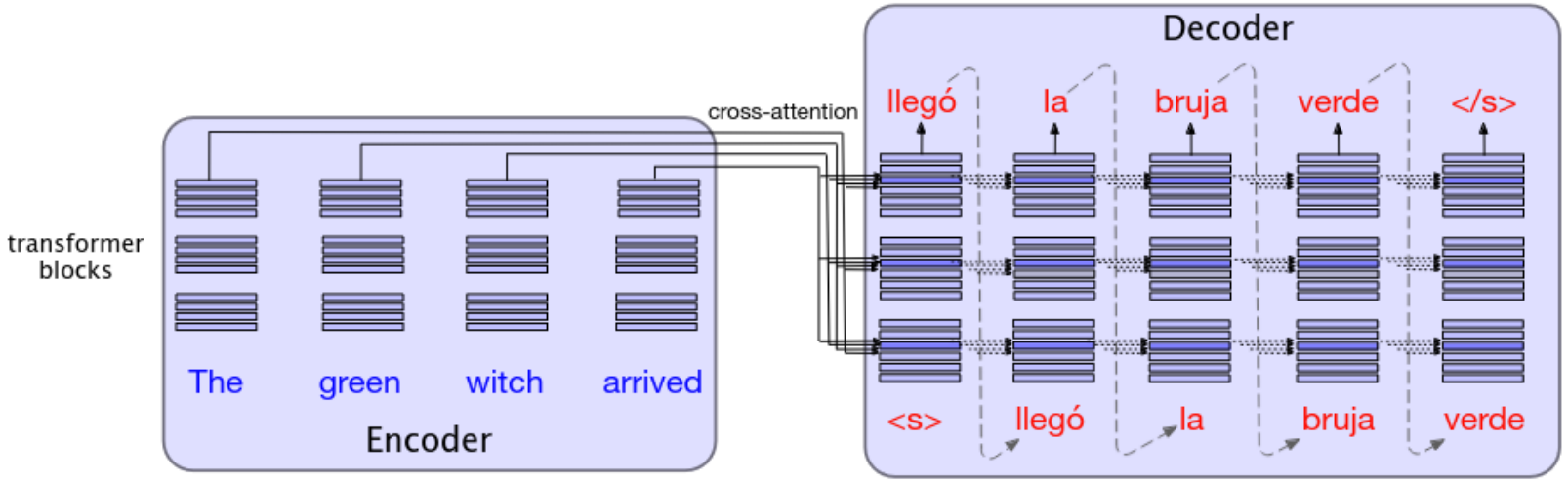
- BERT: After pretraining, *fine-tune* model for language understanding tasks by adding a classification head
- Train model as a classifier but initialize parameters with pretrained model



Jurafsky and Martin (2024) Ch. 11

Encoder-decoders

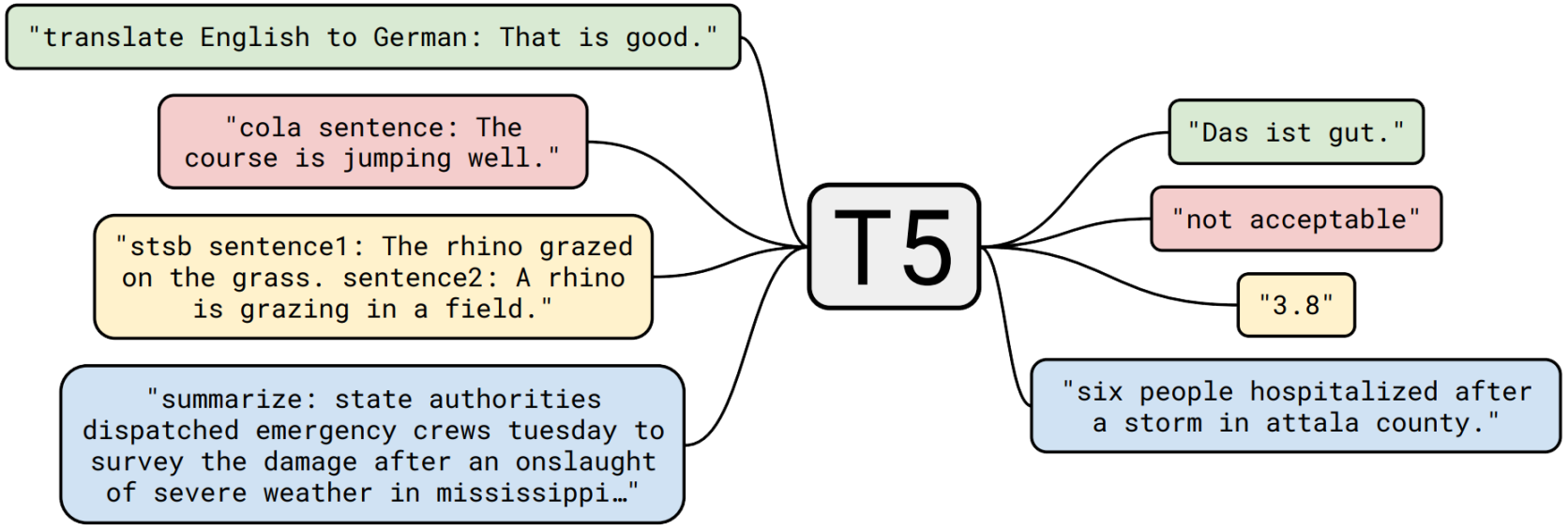
- The original Transformer (Vaswani et al., 2017) was trained for machine translation, with both an encoder and decoder component, connect with cross-attention
- An input sequence is encoded by the encoder, and an output sequence is generated by the decoder



Jurafsky and Martin (2024)

Encoder-decoders

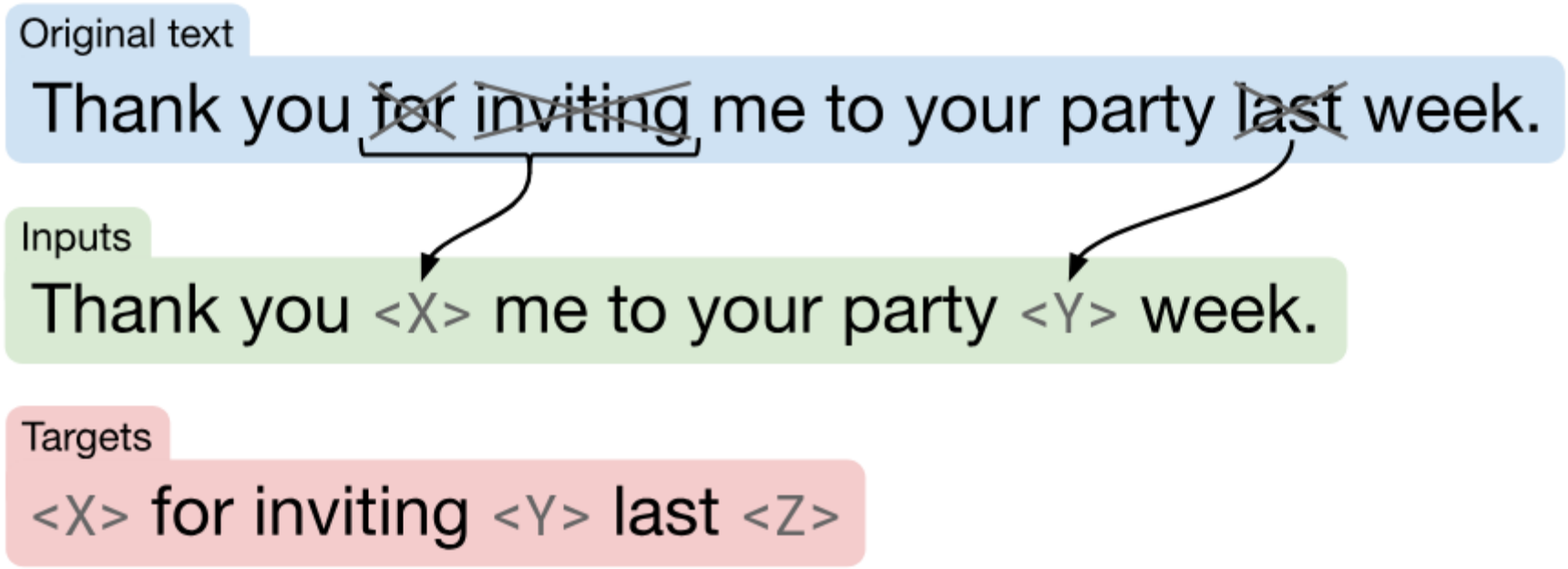
- T5 (Raffel et al., 2019): Encoder-decoder model that was trained to do multiple tasks
  - In each task, the encoder encodes the task name and input, and the decoder generates the output



Raffel et al. (2020)

Encoder-decoders

- T5: Model was pretrained on multiple denoising objectives to predict missing text, such as:

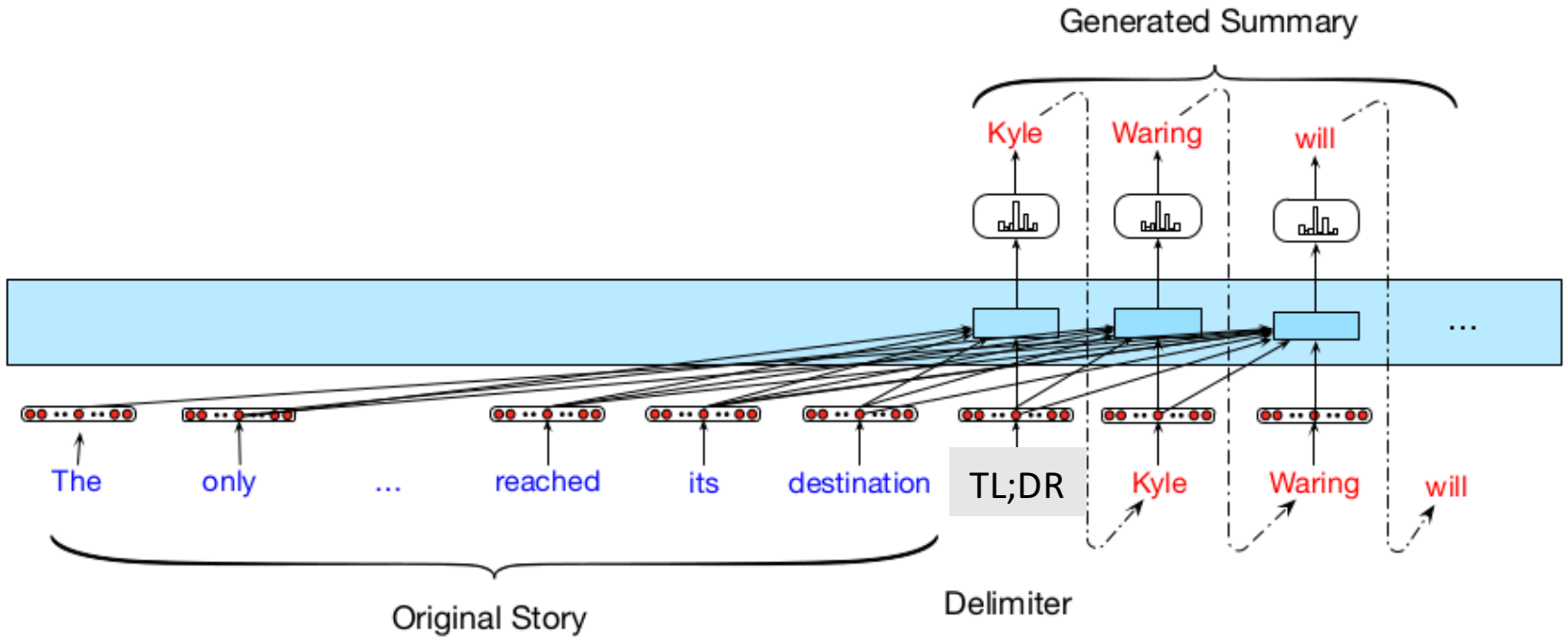


Raffel et al. (2020)

Decoder LLMs

Decoder language models can be applied to text-to-text tasks by letting the LM predict the output as the continuation of the input sequence

- e.g. for summarization using “TLDR:” after the input text



Brown et al. (2020)

Decoder LLMs

Decoder language models can be applied more generally by constructing a *prompt* and letting the LM to predict the output as the continuation

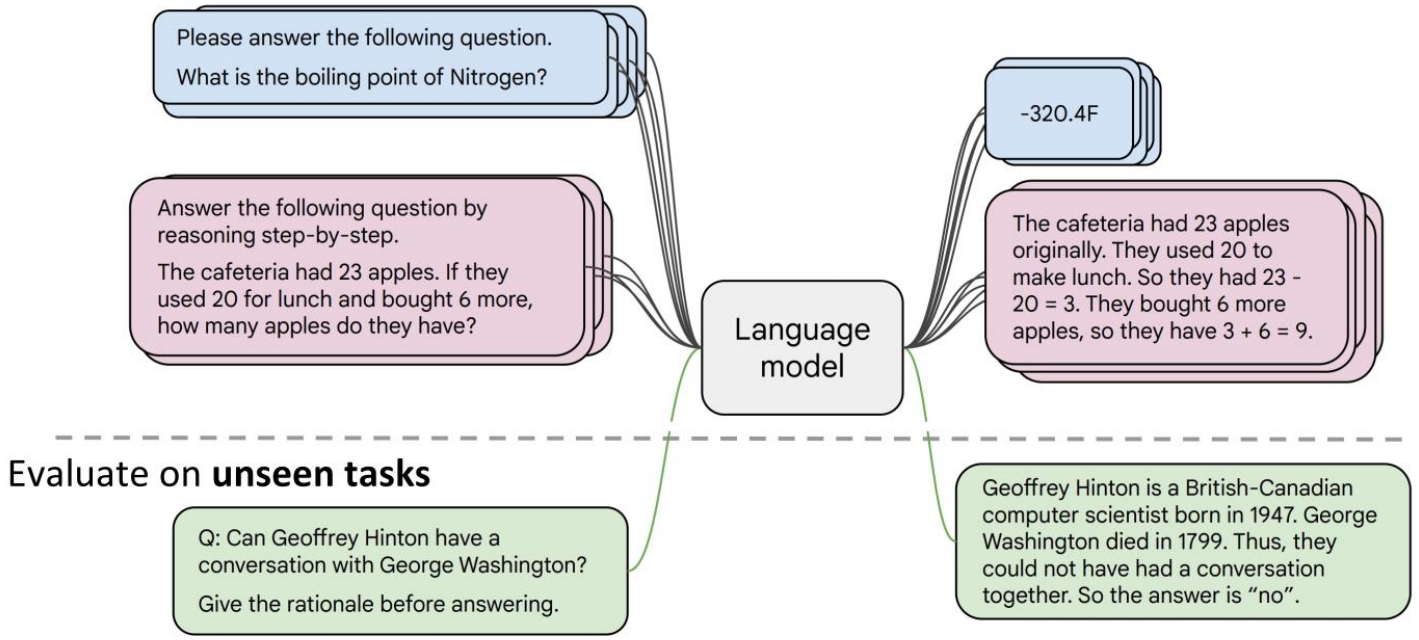
- e.g. for Question Answering

Context →	Q: Who played tess on touched by an angel? A:
Target Completion →	Delloreese Patricia Early (July 6, 1931 { November 19, 2017), known professionally as Della Reese

Brown et al. (2020)

Decoder LLMs

- Decoder LMs can be fine-tuned on multiple tasks (similar to the T5 encoder-decoder)
- At a large scale this leads to the model being able to follow instructions to perform new tasks not including during fine-tuning



Three stages of LLM training

